YT Digest

YouTube Transcript Analyzer — Installation & Reference Guide

Document generated: March 15, 2026

Version: 4.0

Platform: Windows 11 / Python 3.13+

Execution: OpenClaw (Clawdbot) — Scheduled Task (Cron)

This document provides complete instructions for installing, configuring, and running the YT Digest program via OpenClaw (Clawdbot). It covers file placement, dependencies, external configuration, database schema, video filtering, rate limiting, scheduled task setup, and troubleshooting.

Table of Contents

1. Overview
2. Architecture Diagram
3. Prerequisites
4. File Locations & Placement
5. Dependency Installation
6. Configuration (config.ini)
7. Video Filtering & Classification
8. Rate Limiting
9. Database Schema & Storage
10. Program Execution Flow
11. Transcript Retry & Manual Summary
12. OpenClaw Scheduled Task Setup
13. Log File & Housekeeping
14. Troubleshooting
15. Quick Reference Card

1. Overview

YT Digest is a Python script designed to run as a background scheduled task via OpenClaw (Clawdbot). Each execution performs the following steps:

- Fetches the YouTube channel's RSS feed and identifies recent videos
- Filters out Shorts, videos under 30 minutes, and members-only videos
- On first run, looks back 3 days and processes videos chronologically
- Checks if each video has already been processed (via SQLite lookup)
- Fetches transcripts from YouTube when available
- Retries videos without transcripts over 48 hours (9 attempts at 6-hour intervals)
- Enforces 10-minute rate limiting between API calls (tracked on disk)
- Sends transcripts to Google Gemini for top-5 highlights analysis
- Stores the title, dates, transcript, and summary in a SQLite database
- Supports manual transcript import via --summarize for videos with no auto-transcript

The script is idempotent — running it multiple times for the same video has no effect after the first successful processing. All configuration is externalized in a config.ini file under %APPDATA%.

2. Architecture Diagram

The following diagram shows the high-level system architecture and data flow:

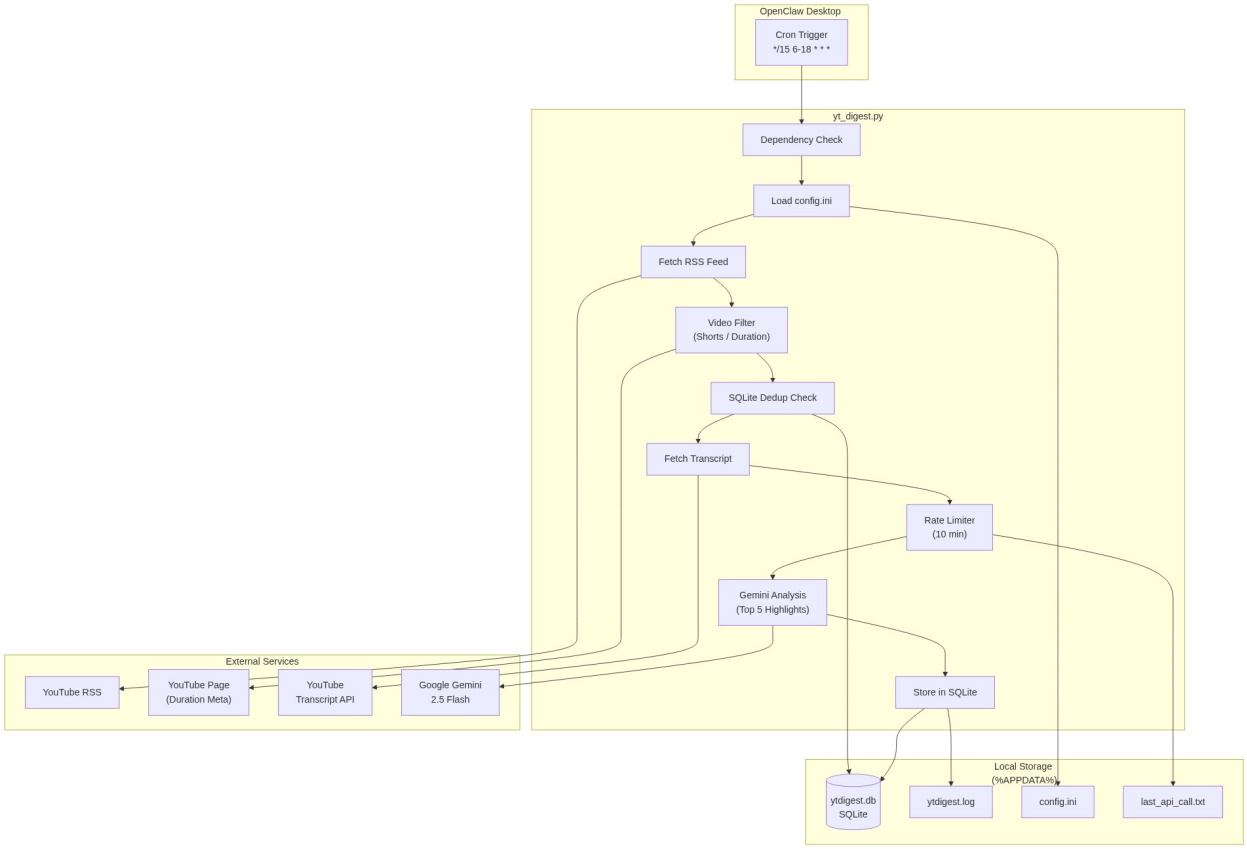

Figure 1: YT Digest System Architecture

3. Prerequisites

3.1 System Requirements

- Windows 10 or 11
- Python 3.10 or later (3.13 recommended) installed and on PATH
- pip (included with Python)
- Internet access (for YouTube RSS, transcript API, and Gemini API)

3.2 API Keys Required

Service	Key Type	Where to Get
`Google Gemini`	API Key	Google AI Studio (aistudio.google.com)
`YouTube`	None required	RSS feed + transcript API — no auth needed

3.3 OpenClaw (Clawdbot)

- OpenClaw Desktop app must be installed
- The app must be **running** for scheduled tasks to fire
- On reboot, relaunch the Desktop app — tasks auto-resume
- Missed runs are caught up (most recent missed run only)

4. File Locations & Placement

All files must be in the correct locations for the system to function.

4.1 Application Directory

All files — the script, requirements, configuration, database, and logs — live together under a single folder in %APPDATA%. This keeps the Desktop clean and works the same on every Windows machine regardless of username.

```
%APPDATA%\YTDigest\
```

Which resolves to: C:\Users\<USERNAME>\AppData\Roaming\YTDigest\

4.2 File Inventory

The directory is created automatically on first run. All files below are stored in %APPDATA%\YTDigest\:

File	Type	Purpose
`yt_digest.py`	Python script	Main application script
`requirements_ytdigest.txt`	Text file	pip dependency manifest (must be alongside yt_digest.py)
`config.ini`	INI file	API keys, channel ID, model settings (auto-created on first run)
`ytdigest.db`	SQLite DB	Processed videos, transcripts, summaries
`ytdigest.log`	Text file	Timestamped execution log
`last_api_call.txt`	Text file	Persistent rate-limit timestamp

NOTE: The requirements file MUST be in the same directory as yt_digest.py. The script looks for it relative to its own location for auto-install.

5. Dependency Installation

5.1 Requirements File Contents

File: requirements_ytdigest.txt

```
feedparser>=6.0.0
requests>=2.25.0
youtube-transcript-api>=1.2.0
google-genai>=1.0.0
```

5.2 Manual Installation

Open a terminal and run:

```
pip install -r %APPDATA%\YTDigest\requirements_ytdigest.txt
```

5.3 Automatic Dependency Check (Built-In)

The script includes a comprehensive startup dependency checker that runs before any third-party imports. On each execution it:

- Verifies each required package is importable
- Checks each package meets the minimum version requirement
- If any package is missing or outdated, attempts **auto-install** from requirements_ytdigest.txt using pip
- Re-verifies all packages after auto-install
- If auto-install fails, prints detailed error messages with exact pip commands and exits with code 1

NOTE: The auto-install requires that requirements_ytdigest.txt is in the same directory as yt_digest.py. If the file is missing, auto-install is skipped and the script exits with instructions for manual installation.

5.4 Dependency Details

Package	Min Version	Purpose
`feedparser`	6.0.0	Parses the YouTube channel RSS/Atom feed to discover recent videos
`requests`	2.25.0	HTTP requests for fetching video page metadata (duration detection)
`youtube-transcript-api`	1.2.0	Fetches auto-generated or manual transcripts from YouTube (no API key needed)
`google-genai`	1.0.0	Google Gemini SDK for sending transcript to Gemini and receiving analysis

5.5 Standard Library Dependencies (No Install Needed)

These are included with Python and require no installation:

- **os, sys, time, re** — System operations and regex
- **sqlite3** — Database operations

- **subprocess** — Running pip for auto-install
- **configparser** — External config.ini parsing
- **importlib.metadata** — Package version checking
- **datetime** — Timestamps, date math, timezone handling
- **pathlib** — Cross-platform file path handling

6. Configuration (config.ini)

All user-configurable settings are stored in an external INI file, not hardcoded in the script. The config file is created automatically on first run with placeholder values.

6.1 Config File Location

```
C:\Users\<USERNAME>\AppData\Roaming\YTDigest\config.ini
```

6.2 Config File Format

```
[ytdigest]
youtube_channel_id = REPLACE_WITH_CHANNEL_ID
gemini_api_key = REPLACE_WITH_GEMINI_API_KEY
gemini_model = gemini-2.5-flash
db_retention_days =
max_log_lines = 500
```

6.3 Settings Reference

Setting	Default	Description
`youtube_channel_id`	REPLACE_WITH_CHANNEL_ID	YouTube channel ID (starts with UC, 24 chars). Must be set before first run.
`gemini_api_key`	REPLACE_WITH_GEMINI_API_KEY	Google Gemini API key from AI Studio. Must be set before first run.
`gemini_model`	gemini-2.5-flash	Gemini model ID. Use gemini-2.5-flash for best speed/cost balance.
`db_retention_days`	(empty = forever)	Days to keep DB records. Leave empty for permanent storage, or set a number (e.g. 365).
`max_log_lines`	500	Log file trimmed to this many lines on each run.

6.4 Changing the YouTube Channel

To monitor a different channel:

- Find the channel ID using: **yt-dlp --print channel_id --playlist-items 1 "https://www.youtube.com/@ChannelHandle"**
- Edit config.ini and replace the youtube_channel_id value
- The channel ID always starts with **UC** followed by 22 characters

6.5 Changing the Gemini Model

Available models as of March 2026:

- **gemini-2.5-flash** — Fast, cost-effective (recommended)
- **gemini-2.5-flash-lite** — Even faster/cheaper for high volume
- **gemini-2.5-pro** — Higher quality analysis, slower, more expensive

NOTE: gemini-2.0-flash is being retired June 1, 2026. Do not use it.

7. Video Filtering & Classification

Not every video on a channel is suitable for transcript analysis. YT Digest uses a multi-layered filtering pipeline to identify only long-form content (30 minutes and up).

7.1 Filter Pipeline

Each video from the RSS feed is classified in this order:

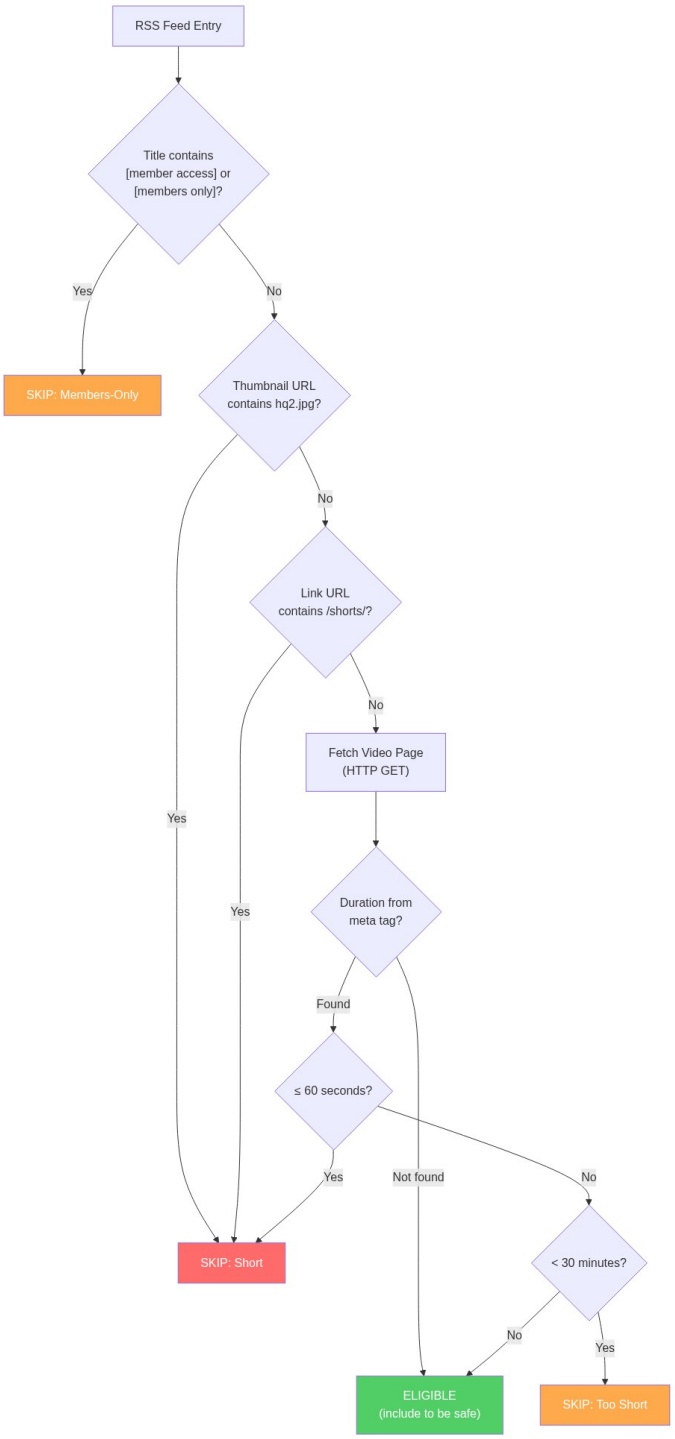

Figure 2: Video Classification Pipeline

7.2 Classification Details

Classification	Criteria	Action
`members-only`	Title contains [member access] or [members only]	Skipped — no accessible transcript
`short`	hq2.jpg thumbnail, /shorts/ URL, or duration ≤ 60s	Skipped — not suitable for transcript analysis

`too_short`	Duration < 30 minutes (1800 seconds)	Skipped — below minimum length threshold
`eligible`	Duration 30 min or longer, or duration unknown	Processed — transcript fetched and analyzed

Duration is extracted from the video page's HTML meta tag: **<meta itemprop="duration" content="PT1H30M45S">** (ISO 8601 duration format).

7.3 Lookback Window

On each run, the script checks the RSS feed for videos published in the last **3 days** (LOOKBACK_DAYS = 3). Videos are processed in chronological order (oldest first). This ensures that on first run or after an outage, recent videos are caught up.

8. Rate Limiting

To avoid hitting YouTube's API quotas, YT Digest enforces a **10-minute minimum interval** between transcript fetch calls. Gemini calls are not rate-limited (free tier allows 15 requests/minute).

8.1 How It Works

- After each transcript fetch, the current Unix timestamp is written to **last_api_call.txt**
- Before the next transcript fetch, the script reads this file and calculates elapsed time
- If less than 10 minutes have passed, the script **sleeps** for the remaining time
- The timestamp file persists across runs — rate limiting works even if the script is restarted

8.2 Rate Limit Timing

Event	Rate Limit Applied
`Transcript fetch`	Yes — waits if < 10 min since last call
`Gemini analysis`	No — Gemini free tier allows 15 req/min
`RSS feed fetch`	No — lightweight, no rate limit needed
`Video page fetch (duration)`	No — simple HTTP GET

NOTE: Each video requires one rate-limited call (transcript fetch) plus one un-limited call (Gemini). Effective per-video time is ~10 minutes when processing back-to-back.

9. Database Schema & Storage

9.1 Database File

Location: C:\Users\<USERNAME>\AppData\Roaming\YTDigest\ytdigest.db

Engine: SQLite 3 (built into Python, no external server required)

9.2 Table: processed_videos

Column	Type	Description
`video_id`	TEXT (PK)	YouTube video ID (e.g., qdRrvIbvdRQ). Primary key — prevents duplicates.
`title`	TEXT	Video title as it appears on YouTube
`published`	DATETIME	Original publish date from YouTube RSS feed (ISO 8601 format)
`processed_at`	DATETIME	Timestamp when YT Digest processed this video (ISO 8601 format)
`status`	TEXT	done, pending_transcript, or no_transcript (see section 9.3)
`summary`	TEXT	Gemini's top 5 highlights analysis (full text stored in DB)
`transcript`	TEXT	Full transcript with timestamps, e.g., [01:23] Hello world
`retry_count`	INTEGER	Number of transcript fetch attempts (used for pending_transcript retries)

NOTE: Summaries are stored directly in the database — no separate summary files are written to disk. The **published** and **processed_at** columns use DATETIME type for proper date sorting and comparison.

9.3 Video Statuses

Each video record has a status field that controls processing behavior:

Status	Meaning	Behavior
`done`	Transcript fetched and Gemini highlights generated	Skipped on future runs
`pending_transcript`	No transcript available yet	Retried automatically every 6 hours (up to 9 attempts over ~48 hours)
`no_transcript`	Gave up after 9 retry attempts	Permanently skipped. Use --summarize to manually import a transcript.

9.4 Schema Migration

The script includes automatic schema migration. If the database was created by an earlier version (before transcript/summary/status/retry_count columns were added), ALTER TABLE is run automatically. No manual intervention is needed.

9.5 Querying the Database

You can inspect the database at any time using the sqlite3 CLI or any SQLite browser (e.g., DB Browser for SQLite):

```
sqlite3 "%APPDATA%\YTDigest\ytdigest.db"
```

Useful queries:

```
-- List all processed videos
SELECT video_id, title, published, processed_at
FROM processed_videos ORDER BY published DESC;

-- Get the latest summary
SELECT title, summary FROM processed_videos
ORDER BY processed_at DESC LIMIT 1;

-- Get a specific transcript
SELECT transcript FROM processed_videos
WHERE video_id = 'VIDEO_ID_HERE';

-- Count total processed videos
SELECT COUNT(*) FROM processed_videos;

-- Videos processed in the last 7 days
SELECT title, processed_at FROM processed_videos
WHERE processed_at >= datetime('now', '-7 days');

-- Videos awaiting transcript
SELECT title, retry_count FROM processed_videos
WHERE status = 'pending_transcript';

-- Videos that gave up (candidates for --summarize)
SELECT video_id, title FROM processed_videos
WHERE status = 'no_transcript';
```

9.6 Retention

db_retention_days is set to empty (keep forever) by default. All records (transcripts, summaries, titles, dates) are kept permanently. To enable pruning, set db_retention_days in config.ini to the desired number of days.

10. Program Execution Flow

The following diagram shows the complete execution flow for each run:

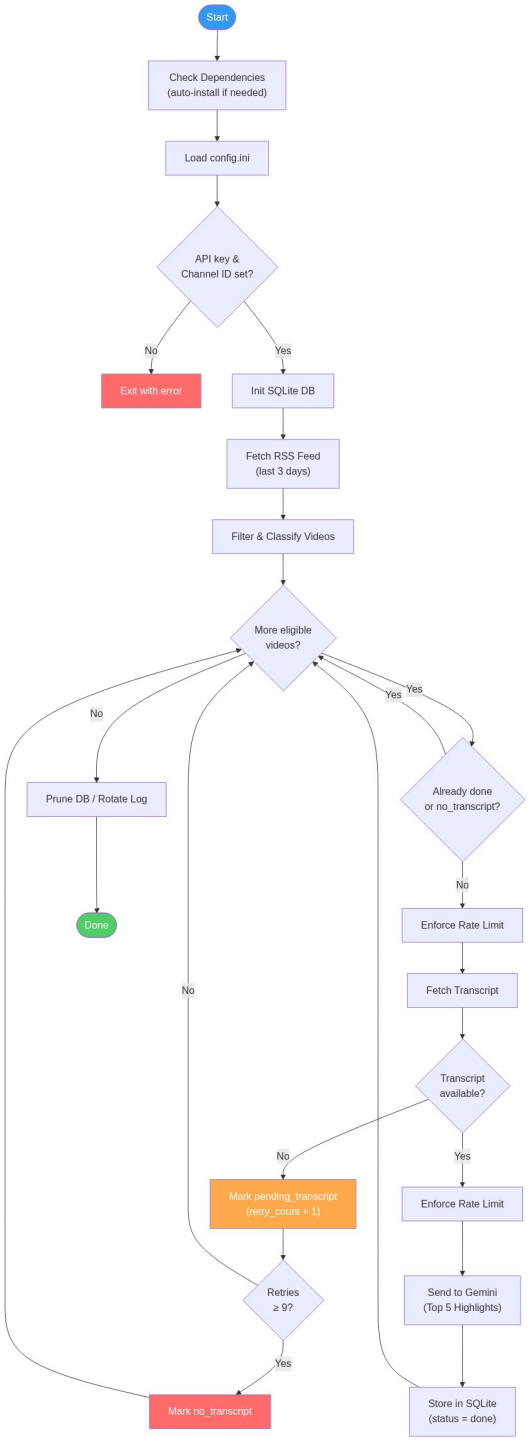

Figure 3: Complete Execution Flow

10.1 Exit Conditions (Normal)

The script exits cleanly (code 0) in these cases:

- No eligible videos found in the lookback window
- All eligible videos already processed (status = done or no_transcript)
- Transcript not yet available — marked as pending_transcript for retry on next run
- Successful processing complete

10.2 Exit Conditions (Error)

The script exits with code 1 in these cases:

- youtube_channel_id is still set to placeholder in config.ini
- gemini_api_key is still set to placeholder in config.ini
- Dependency check failed and auto-install failed

11. Transcript Retry & Manual Summary

11.1 Automatic Retry (pending_transcript)

YouTube can take up to 24 hours (or more) to generate auto-captions for a newly uploaded video. When a video is discovered but has no transcript, the script marks it as **pending_transcript** and retries on subsequent runs.

- First run: video discovered, transcript fetch fails → status = pending_transcript, retry_count = 1
- Every 6 hours: cron runs again, retries pending_transcript videos
- After 9 attempts (~48 hours): status flips to **no_transcript** (gives up)

The retry_count column tracks how many attempts have been made. Videos with status = pending_transcript are retried automatically — no user action needed.

11.2 Manual Summary (--summarize)

If a video ends up as **no_transcript** (or if you have your own transcript from Whisper or another tool), you can manually import it:

```
python yt_digest.py --summarize VIDEO_ID path\to\transcript.txt
```

This command:

- Reads the transcript from the specified text file
- Looks up the video's title and publish date from the existing DB record
- Sends the transcript to Gemini for top-5 highlights analysis
- Marks the video as **done** with the transcript and summary stored in the database

NOTE: The --summarize command bypasses the instance lock and can be run at any time, even while a cron job is active. Rate limiting still applies to the Gemini call.

11.3 Example Workflow

Typical lifecycle for a video without auto-captions:

- Run 1 (00:00): Video discovered, no transcript → pending_transcript (1/9)
- Run 2 (06:00): Retry → still no transcript (2/9)
- Run 3 (12:00): Retry → transcript appears! → Gemini analysis → done

Or, if transcripts never appear:

- Runs 1-9 over 48 hours: all fail → status flips to no_transcript
- User runs Whisper on the video and produces transcript.txt
- User runs: python yt_digest.py --summarize VIDEO_ID transcript.txt
- Video is now status = done with transcript and summary in the database

12. OpenClaw Scheduled Task Setup

12.1 Overview

The script is executed by OpenClaw (Clawdbot) as a scheduled task (cron job). OpenClaw has its own built-in scheduler — it does NOT use Windows Task Scheduler. The Desktop app must be running for tasks to fire.

12.2 Creating the Scheduled Task

In OpenClaw Desktop:

- Open the Desktop app
- Navigate to the Schedule feature
- Create a new scheduled task named **"yt-digest"**
- Set the schedule to run every 6 hours
- Set the prompt/instruction to:

```
Run the command: python %APPDATA%\YTDigest\yt_digest.py
```

Alternatively, the task is stored on disk and can be edited directly:

```
~/.claude/scheduled-tasks/yt-digest/SKILL.md
```

12.3 Recommended Cron Schedule

Run every 6 hours to catch new videos and retry pending transcripts:

```
0 */6 * * *
```

This means: at the top of the hour, every 6 hours (midnight, 6 AM, noon, 6 PM), every day. This schedule allows up to 9 retries over 48 hours for pending transcripts.

> **NOTE:** The script is idempotent. Once a video is processed, subsequent runs for the same video exit immediately with 'already processed'. There is no penalty for running it frequently.

12.4 What Happens on Reboot

- Scheduled tasks are **persistent** — stored on disk, survive restarts
- When the Desktop app starts, it auto-resumes all scheduled tasks
- It catches up the **most recent missed run** (checks last 7 days)
- Older missed runs are discarded (no queue of 50+ runs after long outage)
- A notification appears when a catch-up run starts

12.5 Requirements for Scheduled Tasks to Run

- OpenClaw Desktop app must be **running**
- Computer must be **awake** (not sleeping/hibernating)
- Consider enabling 'Keep computer awake' in OpenClaw Desktop settings

- Internet connection must be available

13. Log File & Housekeeping

13.1 Log File

Location: C:\Users\<USERNAME>\AppData\Roaming\YTDigest\ytdigest.log

Format: [YYYY-MM-DD HH:MM:SS] message

The log captures every action: RSS fetch, video classification, transcript status, rate limiting waits, Gemini calls, errors, and housekeeping. It is both printed to console and written to file.

Example log entries:

```
[2026-03-12 09:15:00] ============================================================
[2026-03-12 09:15:00] YT Digest run started.
[2026-03-12 09:15:00] Fetching RSS feed for channel: UCZk3...
[2026-03-12 09:15:01] Eligible videos: 2 (skipped 8 shorts, 3 too short, 0
members-only)
[2026-03-12 09:15:01] Checking: "Deep Dive into Transformers" (abc123)
[2026-03-12 09:15:01] Fetching transcript for video: abc123
[2026-03-12 09:15:02] Transcript retrieved: 24523 characters
[2026-03-12 09:15:02] Rate limit: waiting 8.3 minutes before next API call...
[2026-03-12 09:23:22] Sending transcript to Gemini for analysis...
[2026-03-12 09:23:28] Summary stored in database.
[2026-03-12 09:23:28] Successfully processed: "Deep Dive into Transformers"
[2026-03-12 09:23:28] Done. Processed: 1, Skipped: 1, Total checked: 2.
[2026-03-12 09:23:28] YT Digest run complete.
```

13.2 Log Rotation

The log file is trimmed to the last 500 lines at the start of each run (configurable via max_log_lines in config.ini). This prevents unbounded growth while keeping enough history for debugging.

14. Troubleshooting

14.1 Script Won't Run — ModuleNotFoundError

The built-in dependency checker should handle this automatically. If it fails:

```
pip install -r %APPDATA%\YTDigest\requirements_ytdigest.txt
```

NOTE: Make sure you're using the same Python that OpenClaw uses. Check with: python --version

14.2 'Please set youtube_channel_id / gemini_api_key in config.ini'

Edit the config file and replace the placeholder values:

```
notepad "%APPDATA%\YTDigest\config.ini"
```

14.3 'No Videos Found in RSS Feed'

- Verify the channel ID is correct in config.ini
- Check internet connectivity
- YouTube may be temporarily down or rate-limiting
- Try opening the feed URL in a browser

14.4 'Transcript Not Yet Available'

This is **normal** for recently uploaded videos. YouTube takes time to generate auto-captions (typically 15 minutes to several hours). The video will be marked as **pending_transcript** and retried automatically every 6 hours for up to 48 hours (9 attempts). After that, it becomes **no_transcript** and you can use --summarize to manually import one.

14.5 'Gemini Analysis Failed'

- Check that gemini_api_key is valid and not expired in config.ini
- Check Gemini API quotas at aistudio.google.com
- The model name may have changed — verify gemini_model is current
- Network issues can cause timeouts — the next run will retry

14.6 Scheduled Task Not Firing

- Is OpenClaw Desktop app running?
- Is the computer awake (not sleeping)?
- Check the task is enabled in the Desktop app's schedule settings
- Review ~/.claude/scheduled-tasks/yt-digest/SKILL.md exists

14.7 Database Locked Error

SQLite can only handle one writer at a time. If two runs overlap:

- The instance lock should prevent this (PID-based lock file)

- If it does happen, the script will fail and retry on the next run
- Check for orphaned Python processes: tasklist | findstr python

14.8 Resetting the Database

To start fresh (reprocess all videos from the last 3 days):

```
del "%APPDATA%\YTDigest\ytdigest.db"
```

The script will create a new database on the next run.

14.9 Resetting the Rate Limiter

If the rate limiter is blocking and you want to force an immediate run:

```
del "%APPDATA%\YTDigest\last_api_call.txt"
```

15. Quick Reference Card

File Locations

```
App Directory: %APPDATA%\YTDigest\
Script: %APPDATA%\YTDigest\yt_digest.py
Requirements: %APPDATA%\YTDigest\requirements_ytdigest.txt
Config: %APPDATA%\YTDigest\config.ini
Database: %APPDATA%\YTDigest\ytdigest.db
Log: %APPDATA%\YTDigest\ytdigest.log
Rate Limiter: %APPDATA%\YTDigest\last_api_call.txt
Cron Task: ~/.claude/scheduled-tasks/yt-digest/SKILL.md
```

Key Commands

```
# Run manually
python %APPDATA%\YTDigest\yt_digest.py

# Manually summarize a video with your own transcript
python yt_digest.py --summarize VIDEO_ID transcript.txt

# Install dependencies
pip install -r %APPDATA%\YTDigest\requirements_ytdigest.txt

# Edit configuration
notepad "%APPDATA%\YTDigest\config.ini"

# Query the database
sqlite3 "%APPDATA%\YTDigest\ytdigest.db" \
"SELECT title, status, processed_at FROM processed_videos;"

# Check pending retries
sqlite3 "%APPDATA%\YTDigest\ytdigest.db" \
"SELECT video_id, title, retry_count FROM processed_videos WHERE
status='pending_transcript';"

# Reset (delete database to reprocess all)
del "%APPDATA%\YTDigest\ytdigest.db"

# Check log
type "%APPDATA%\YTDigest\ytdigest.log"
```

Configuration At-a-Glance

```
Config File: %APPDATA%\YTDigest\config.ini
Gemini Model: gemini-2.5-flash
DB Retention: Forever (empty)
Log Rotation: 500 lines
Rate Limit: 10 min between API calls
Lookback: 3 days
Min Duration: 30 minutes
Max Duration: None (no upper limit)
Cron Schedule: Every 6 hours (0 */6 * * *)
```

Video Filtering Summary

```
SKIP: YouTube Shorts (hq2.jpg thumbnail or /shorts/ URL)
SKIP: Videos under 60 seconds (detected Shorts)
SKIP: Videos under 30 minutes (too short)
SKIP: Members-only (title pattern match)
KEEP: Videos 30 min and up (no upper limit)
```

End of document. Generated March 15, 2026 at 11:31 AM.